

ArcSettle

Reserve-gated USDC settlement with on-chain proof-of-reserves, on Arc

A settlement rail whose every settlement provably cannot exceed the USDC reserves that have actually been proven.

Milestone 1 of the ArcSettle grant proposal. Arc testnet only.

doom2quake · Dipankar Sarkar

The gap that costs money

Fiat-backed stablecoins settle **on-chain** but prove solvency **off-chain**.

- The chain sees tokens move. It does not see whether they are backed **right now**.
- When an attestation lags, or a redemption frees headroom before the outflow is seen, supply outruns reserves.
- Whoever accepts the token as settlement wears that gap, and cannot see it until it has already cost them.

It gets worse with an agent on the treasury

An autonomous agent that settles on a schedule will, unless something stops it:

- settle against whatever reserve figure it **last read**,
- waste gas on transactions that revert,
- or act on a figure a redemption has **already invalidated**.

"We never settle past proven reserves" must be an on-chain **invariant**,
not a line in a runbook.

Why Arc

On **Arc**, Circle's settlement chain:

- the **reserve asset**, the **settled asset**, and **gas** are the same regulated dollar (USDC);
- the invariant "circulating never exceeds attested reserves" is one unit, end to end;
- no bridge seam sits in the middle of the guarantee.

On a general-purpose chain this is a bolt-on. On Arc it is native to how value moves.

The invariant

On every settlement:

```
totalSupply + amount <= effectiveReserves
```

```
effectiveReserves = 0           if attestation stale or future-dated  
                  = max(0, R - D) otherwise
```

- **R** = attested USDC reserves (EIP-712 signed proof)
- **D** = redemption debt: USDC redeemed the current attestation has not yet seen

The hero defence: redemption does not reopen headroom

Naive reserve gates let a redemption reopen capacity the instant supply falls.

The attack: **redeem → move the freed USDC out → re-settle against the same old proof.**

ArcSettle books the freed amount as `pendingRedemptions` and nets it off reserves until a **new** attestation has observed the outflow. Headroom stays shut until then.

`test_RedeemThenResettleAgainstOldAttestationReverts` goes red if you delete it.

Two contracts, one file each

ReserveAttestationOracle.sol : reserves enter only against an EIP-712 signature from a registered attester. On-chain recovery; replays, malleable / bad-length / bad-v signatures, backdated and future-dated proofs all refused. Adding an attester is timelocked; revoking is immediate.

ArcSettlement.sol : reverts **InsufficientReserves** / **StaleAttestation** / **FutureAttestation** . Oracle and freshness change only through a 2-day timelock; freshness cannot be switched off. Reads the oracle with **STATICCALL** , so it cannot be re-entered.

The operator agent

Runs the on-chain gate **off-chain first**:

- an impossible amount is rejected before anything else,
- a settlement that would exceed reserves is **refused, spending no gas**,
- only a settlement that will actually be submitted spends action quota,
- every decision is written to an audit trail (agent-core `ActionLimiter`).

In live mode the pre-check is a real `eth_call` dry-run; the revert reason is the **contract's own custom error**, not a guess.

Verified, not asserted

- **59 Solidity tests** (`forge test` , solc 0.8.24): 36 on the gate, 23 on the oracle.
- **75 Python tests** (`pytest`): agent, ABI layer, live-mode config and revert decoding.

Every defence has a test that fails without it. The Python suite mirrors the Solidity suite test-for-test, so they read side by side.

```
$ forge test    -> 59 passed  
$ pytest tests -> 75 passed
```

Honest limits (stated plainly)

- **No mainnet**, none planned under this grant. Testnet only.
- **No live broadcast** yet, reads and dry-runs only. That is milestone 3.
- **No users. No revenue. No audit. No partnership with Circle.**

Our substitute for traction is mutation-tested code and a candid scope map.

See `docs/LIMITATIONS.md` .

Roadmap

1. **M1 (this)**: reserve-gate contract + EIP-712 oracle, ported to Arc, USDC-denominated, tested.
2. **M2**: Arc testnet deployment + live read path against the deployed address.
3. **M3**: agentic settlement broadcast with guardrails + a Circle-primitive custody seam.
4. **M4**: reusable, documented proof-of-reserves module for any Arc USDC contract.

The durable contribution: a standard, tested answer to how an agent settles in USDC on Arc without ever exceeding proven reserves, established before mainnet.